

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA DE SOFTWARE

APLICACIONES DISTRIBUIDAS (ISR-701)

TEMA:

DISEÑO E IMPLEMENTACIÓN DE UN PROTOTIPO DE
BASE DE DATOS DISTRIBUIDA CON FRAGMENTACIÓN
HORIZONTAL, ENRUTAMIENTO DINÁMICO Y
TOLERANCIA A FALLOS PARA EL BANCO DEL AUSTRO

Informe académico de práctica de laboratorio

Autora:

Keyla Bedón Viteri

Docente:

Ph.D. Gleiston Guerrero Ulloa

Período académico:

2026 – 2027

Quevedo – Ecuador
Julio, 2026

1. Introducción

Las bases de datos distribuidas (BDD) son hoy un pilar del sector financiero, donde la disponibilidad continua, la baja latencia y la integridad transaccional son requisitos ineludibles. Un banco con oficinas en varias ciudades, como el caso del Banco del Austro, no puede centralizar sus datos en un único servidor sin sacrificar el tiempo de respuesta local y sin crear un punto único de falla que afecte a todas las sedes a la vez. Este informe documenta el diseño, la implementación y la verificación de un prototipo de BDD para el Banco del Austro, construido sobre tres nodos PostgreSQL independientes (Cuenca, Quito y Guayaquil) orquestados por una capa de enrutamiento en Spring Boot 3 / Java 21. El prototipo extiende la guía de práctica de dos a tres sedes, incorpora un endpoint transaccional de transferencias inter-sede y una prueba de tolerancia a fallos, cumpliendo los objetivos de la Unidad 2 de Aplicaciones Distribuidas (ISR-701).

2. Diseño de la Arquitectura Distribuida

El sistema implementa una arquitectura de tres capas claramente desacopladas, cada una con una responsabilidad única, lo que favorece la mantenibilidad y permite sustituir cualquiera de las capas sin afectar a las demás:

- **Capa Cliente:** compuesta por Postman y, alternativamente, cualquier navegador web. Esta capa es responsable exclusivamente de emitir peticiones HTTP (GET y POST) y de interpretar las respuestas JSON. El cliente desconoce por completo cuántos nodos de base de datos existen detrás del servicio; solo interactúa con un único endpoint expuesto en el puerto 8080.
- **Capa Middleware / Router:** una aplicación Spring Boot 3 ejecutada sobre Java 21, que concentra toda la lógica de enrutamiento, la unión de fragmentos y la orquestación de transacciones distribuidas. Esta capa es el corazón arquitectónico del prototipo: es aquí donde se materializa la transparencia de fragmentación y la transparencia de ubicación exigidas por la teoría de bases de datos distribuidas.
- **Capa de Infraestructura de Datos:** tres contenedores Docker independientes, cada uno ejecutando una imagen oficial `postgres:16-alpine`, con su propio volumen persistente y su propio esquema de datos. Cada contenedor representa un nodo autónomo que puede iniciarse, detenerse o fallar sin afectar directamente el estado de los otros dos.

2.1 Distribución geográfica del sistema

La distribución física-lógica de los nodos, junto con la convención de prefijos que habilita el enrutamiento determinista por número de cuenta, se resume en la siguiente tabla:

Nodo (Sede)	Puerto	Prefijo	Base de datos
Cuenca (sede central)	5432	22	banco_cuenca
Quito	5433	17	banco_quito
Guayaquil	5434	09	banco_guayaquil

Table 1: Distribución de nodos, puertos y prefijos de enrutamiento

Cada nodo escucha internamente en el puerto estándar de PostgreSQL (5432), pero Docker remapea ese puerto interno a un puerto distinto en el host para evitar colisiones, lo que permite que los tres motores coexistan en la misma máquina de desarrollo simulando tres ubicaciones físicas separadas. El enrutador de la aplicación Java interpreta los dos primeros dígitos del

número de cuenta como una clave de directorio: no consulta ninguna tabla de metadatos para decidir a qué nodo dirigirse, sino que aplica una función determinista que traduce directamente el prefijo en el bean de conexión correspondiente (`dsCuenca`, `dsQuito` o `dsGuayaquil`). Esta decisión de diseño, discutida en la guía de práctica, prioriza la simplicidad didáctica sobre la flexibilidad operativa; en un sistema de producción el enrutamiento se apoyaría en una tabla de directorio dinámica o en un esquema de hashing consistente para no acoplar la lógica de negocio a una convención de nomenclatura de cuentas.

3. Tecnologías Utilizadas

Docker Compose: permite levantar los tres motores de PostgreSQL con un solo comando, sin instalar nada de forma local. Facilita crear o eliminar toda la infraestructura (contenedores, volúmenes y red) de manera rápida y repetible.

PostgreSQL 16 Alpine: se usó por ser una versión liviana y rápida de descargar. Permite aplicar restricciones CHECK sobre la columna oficina, obligando a cada nodo a guardar solo los datos que le corresponden según la fragmentación horizontal.

Spring Boot 3 / Java 21: framework encargado de la capa intermedia. Se usó JdbcTemplate para ejecutar las consultas SQL de forma directa, y la anotación @Transactional en el método de transferencias para que Spring controle el commit o rollback automáticamente.

Postman: herramienta utilizada para probar cada endpoint, revisar los códigos de respuesta HTTP y comparar los tiempos entre consultas locales e inter-sede.

4. Implementación: Código Fuente

A continuación se presenta el código fuente completo del prototipo, organizado según la estructura de paquetes utilizada en el proyecto Maven/Spring Boot.

4.1 Infraestructura (docker-compose.yml)

El siguiente archivo declara los tres servicios de base de datos, cada uno con su propio volumen persistente y su propio mapeo de puertos, conectados a una red interna común (`bda-net`):

`docker-compose.yml`

```
services:
  db-cuenca:
    image: postgres:16-alpine
    container_name: banco-cuenca
    environment:
      POSTGRES_DB: banco_cuenca
      POSTGRES_USER: banco
      POSTGRES_PASSWORD: banco123
    ports:
      - "5432:5432"
    volumes:
      - vol-cuenca:/var/lib/postgresql/data
      - ./sql-cuenca:/docker-entrypoint-initdb.d
    networks:
      - bda-net

  db-quito:
    image: postgres:16-alpine
    container_name: banco-quito
    environment:
      POSTGRES_DB: banco_quito
      POSTGRES_USER: banco
      POSTGRES_PASSWORD: banco123
```

```

ports:
  - "5433:5432"
volumes:
  - vol-quito:/var/lib/postgresql/data
  - ./sql-quito:/docker-entrypoint-initdb.d
networks:
  - bda-net

db-guayaquil:
  image: postgres:16-alpine
  container_name: banco-guayaquil
  environment:
    POSTGRES_DB: banco_guayaquil
    POSTGRES_USER: banco
    POSTGRES_PASSWORD: banco123
  ports:
    - "5434:5432"
  volumes:
    - vol-guayaquil:/var/lib/postgresql/data
    - ./sql-guayaquil:/docker-entrypoint-initdb.d
  networks:
    - bda-net

networks:
  bda-net:
    driver: bridge

volumes:
  vol-cuenca:
  vol-quito:
  vol-guayaquil:

```

4.2 Configuración de datos (sql-guayaquil como ejemplo)

El esquema del nodo Guayaquil ilustra el mecanismo de fragmentación horizontal: la restricción CHECK sobre la columna oficina impide, a nivel de motor, que se inserte en este nodo una fila perteneciente a cualquier otra sede, garantizando así la propiedad de disjunción de los fragmentos.

[sql-guayaquil/01_schema.sql](#)

```

CREATE TABLE clientes (
  cedula VARCHAR(10) PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  ciudad VARCHAR(50) NOT NULL
);

CREATE TABLE cuentas (
  numero VARCHAR(20) PRIMARY KEY,
  saldo NUMERIC(12, 2) NOT NULL,
  oficina VARCHAR(50) NOT NULL,
  CONSTRAINT chk_oficina CHECK (oficina = 'GUAYAQUIL')
);

```

[sql-guayaquil/02_datos.sql](#)

```

INSERT INTO clientes (cedula, nombre, ciudad) VALUES
('0900000001', 'Juan Pueblo', 'Guayaquil'),
('0900000002', 'Mafer Andrade', 'Guayaquil');

INSERT INTO cuentas (numero, saldo, oficina) VALUES
('0901000001', 350.00, 'GUAYAQUIL'),
('0901000002', 1500.75, 'GUAYAQUIL');

```

4.3 Propiedades de conexión (application.properties)

Las credenciales y URLs JDBC de los tres nodos se externalizan en `application.properties`, permitiendo que la clase de configuración las inyecte en tiempo de ejecución sin acoplar el código Java a valores fijos:

`src/main/resources/application.properties`

```
server.port=8080
spring.application.name=banco-austro

# Configuración Base de Datos Cuenca
datasources.cuenca.url=jdbc:postgresql://localhost:5432/banco_cuenca
datasources.cuenca.username=banco
datasources.cuenca.password=banco123
datasources.cuenca.driver-class-name=org.postgresql.Driver

# Configuración Base de Datos Quito
datasources.quito.url=jdbc:postgresql://localhost:5433/banco_quito
datasources.quito.username=banco
datasources.quito.password=banco123
datasources.quito.driver-class-name=org.postgresql.Driver

# Configuración Base de Datos Guayaquil
datasources.guayaquil.url=jdbc:postgresql://localhost:5434/banco_guayaquil
datasources.guayaquil.username=banco
datasources.guayaquil.password=banco123
datasources.guayaquil.driver-class-name=org.postgresql.Driver
```

4.4 Configuración de conexiones (DataSourceConfig.java)

Esta clase, anotada con `@Configuration`, declara tres beans de tipo `DataSource` — `dsCuenca`, `dsQuito` y `dsGuayaquil` — cada uno identificado por nombre para poder ser inyectado selectivamente en la capa de servicio mediante `@Qualifier`:

`config/DataSourceConfig.java`

```
package ec.edu.uteq.bancoaustro.config;

import javax.sql.DataSource;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.jdbc.DataSourceBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class DataSourceConfig {

    @Value("${datasources.cuenca.url}")
    private String cuencaUrl;
    @Value("${datasources.cuenca.username}")
    private String cuencaUser;
    @Value("${datasources.cuenca.password}")
    private String cuencaPass;

    @Value("${datasources.quito.url}")
    private String quitoUrl;
    @Value("${datasources.quito.username}")
    private String quitoUser;
    @Value("${datasources.quito.password}")
    private String quitoPass;

    @Value("${datasources.guayaquil.url}")
    private String guayaquilUrl;
    @Value("${datasources.guayaquil.username}")
    private String guayaquilUser;
    @Value("${datasources.guayaquil.password}")
    private String guayaquilPass;
```

```

@Bean(name = "dsCuenca")
public DataSource dsCuenca() {
    return DataSourceBuilder.create()
        .url(cuencaUrl)
        .username(cuencaUser)
        .password(cuencaPass)
        .driverClassName("org.postgresql.Driver")
        .build();
}

@Bean(name = "dsQuito")
public DataSource dsQuito() {
    return DataSourceBuilder.create()
        .url(quitoUrl)
        .username(quitoUser)
        .password(quitoPass)
        .driverClassName("org.postgresql.Driver")
        .build();
}

@Bean(name = "dsGuayaquil")
public DataSource dsGuayaquil() {
    return DataSourceBuilder.create()
        .url(guayaquilUrl)
        .username(guayaquilUser)
        .password(guayaquilPass)
        .driverClassName("org.postgresql.Driver")
        .build();
}
}

```

4.5 Enrutador y lógica de negocio (ConsultaDistribuidaService.java)

Esta clase concentra las tres capacidades centrales del prototipo: el enrutamiento determinista por prefijo de cuenta, la unión resiliente de fragmentos y la transferencia transaccional entre sedes.

4.5.1 Enrutamiento por prefijo de cuenta

El método privado `enrutar()` traduce el prefijo numérico de la cuenta en el `JdbcTemplate` correspondiente al nodo físico donde reside el fragmento, sin que ninguna otra clase del sistema necesite conocer esta correspondencia.

4.5.2 Unión resiliente de fragmentos con tolerancia a nodos caídos

El método `listarTodosLosClientes()` envuelve cada consulta a un nodo en un bloque `try-catch` independiente. Si un nodo no responde, la excepción se captura, se registra una advertencia en la consola del servidor y la ejecución continúa con los nodos restantes, en lugar de propagar un error 500 que interrumpiría la respuesta completa.

4.5.3 Transferencias transaccionales distribuidas

El método `realizarTransferencia()`, anotado con `@Transactional`, valida primero que la cuenta de origen tenga saldo suficiente, ejecuta el débito en el nodo de origen y, a continuación, el crédito en el nodo de destino, determinando además si la operación es intra-sede o inter-sede comparando las instancias de `JdbcTemplate` involucradas.

`service/ConsultaDistribuidaService.java`

```

package ec.edu.uteq.bancoaustro.service;

import java.util.ArrayList;
import java.util.List;

```

```

import java.util.Map;
import java.util.HashMap;
import javax.sql.DataSource;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class ConsultaDistribuidaService {

    private final JdbcTemplate jdbcCuenca;
    private final JdbcTemplate jdbcQuito;
    private final JdbcTemplate jdbcGuayaquil;

    public ConsultaDistribuidaService(
        @Qualifier("dsCuenca") DataSource dsCuenca,
        @Qualifier("dsQuito") DataSource dsQuito,
        @Qualifier("dsGuayaquil") DataSource dsGuayaquil) {
        this.jdbcCuenca = new JdbcTemplate(dsCuenca);
        this.jdbcQuito = new JdbcTemplate(dsQuito);
        this.jdbcGuayaquil = new JdbcTemplate(dsGuayaquil);
    }

    public Map<String, Object> consultarSaldo(String numero) {
        JdbcTemplate destino = enrutar(numero);
        String sql = "SELECT numero, saldo, oficina FROM cuentas WHERE numero = ?";
        List<Map<String, Object>> filas = destino.queryForList(sql, numero);

        if (filas.isEmpty()) {
            return Map.of("error", "Cuenta no encontrada", "numero", numero);
        }
        return filas.get(0);
    }

    public List<Map<String, Object>> listarTodosLosClientes() {
        String sql = "SELECT cedula, nombre, ciudad FROM clientes";
        List<Map<String, Object>> union = new ArrayList<>();

        try {
            union.addAll(jdbcCuenca.queryForList(sql));
        } catch (Exception e) {
            System.err.println("Advertencia: Nodo CUENCA fuera de linea.");
        }

        try {
            union.addAll(jdbcQuito.queryForList(sql));
        } catch (Exception e) {
            System.err.println("Advertencia: Nodo QUITO fuera de linea.");
        }

        try {
            union.addAll(jdbcGuayaquil.queryForList(sql));
        } catch (Exception e) {
            System.err.println("Advertencia: Nodo GUAYAQUIL fuera de linea.");
        }

        return union;
    }

    @Transactional
    public Map<String, Object> realizarTransferencia(String origen, String destino, double monto) {
        if (monto <= 0) {
            throw new IllegalArgumentException("El monto debe ser mayor a cero.");
        }

        JdbcTemplate dbOrigen = enrutar(origen);
        JdbcTemplate dbDestino = enrutar(destino);

        String sqlSaldo = "SELECT saldo FROM cuentas WHERE numero = ?";
        Double saldoOrigen = dbOrigen.queryForObject(sqlSaldo, Double.class, origen);
        if (saldoOrigen == null || saldoOrigen < monto) {
            throw new IllegalStateException("Saldo insuficiente en la cuenta de origen: " + origen);
        }
    }
}

```

```

    }

    String sqlDebito = "UPDATE cuentas SET saldo = saldo - ? WHERE numero = ?";
    dbOrigen.update(sqlDebito, monto, origen);

    String sqlCredito = "UPDATE cuentas SET saldo = saldo + ? WHERE numero = ?";
    int filasAfectadas = dbDestino.update(sqlCredito, monto, destino);

    if (filasAfectadas == 0) {
        throw new IllegalArgumentException("La cuenta destino no existe: " + destino);
    }

    Map<String, Object> respuesta = new HashMap<>();
    respuesta.put("mensaje", "Transferencia realizada con exito.");
    respuesta.put("origen", origen);
    respuesta.put("destino", destino);
    respuesta.put("monto", monto);
    respuesta.put("tipo", dbOrigen == dbDestino ? "INTRA_SEDE (Local)" : "INTER_SEDE (Distribuida)");

    return respuesta;
}

private JdbcTemplate enrutar(String numero) {
    if (numero == null || numero.length() < 2) {
        throw new IllegalArgumentException("Numero de cuenta invalido: " + numero);
    }
    String prefijo = numero.substring(0, 2);
    return switch (prefijo) {
        case "22" -> jdbcCuenca;
        case "17" -> jdbcQuito;
        case "09" -> jdbcGuayaquil;
        default -> throw new IllegalArgumentException("Prefijo de oficina no reconocido: " + prefijo);
    };
}
}
}

```

4.6 Controlador REST (BancoController.java)

El controlador expone tres endpoints públicos bajo la ruta base `/api/banco`: consulta de saldo por número de cuenta, listado unificado de clientes de las tres sedes, y registro de una transferencia mediante una petición POST cuyo cuerpo JSON incluye origen, destino y monto.

`controller/BancoController.java`

```

package ec.edu.uteq.bancoaustro.controller;

import java.util.List;
import java.util.Map;
import ec.edu.uteq.bancoaustro.service.ConsultaDistribuidaService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/banco")
public class BancoController {

    private final ConsultaDistribuidaService service;

    public BancoController(ConsultaDistribuidaService service) {
        this.service = service;
    }

    @GetMapping("/saldo/{numero}")
    public Map<String, Object> saldo(@PathVariable String numero) {
        return service.consultarSaldo(numero);
    }

    @GetMapping("/clientes")
    public List<Map<String, Object>> clientes() {
        return service.listarTodosLosClientes();
    }
}

```



```
}

@PostMapping("/transferencia")
public Map<String, Object> transferir(@RequestBody Map<String, Object> payload) {
    String origen = (String) payload.get("origen");
    String destino = (String) payload.get("destino");
    double monto = Double.parseDouble(payload.get("monto").toString());

    try {
        return service.realizarTransferencia(origen, destino, monto);
    } catch (Exception e) {
        return Map.of("error", e.getMessage());
    }
}
```

5. Pruebas de Verificación y Resultados

Se diseñaron cuatro pruebas funcionales que verifican, en orden creciente de complejidad, las propiedades de transparencia de fragmentación, enrutamiento dinámico, atomicidad transaccional y tolerancia a fallos del prototipo. Todas las pruebas se ejecutaron con Postman contra la aplicación Spring Boot corriendo en <http://localhost:8080>, con los tres contenedores de PostgreSQL activos.

5.1 Prueba 1: Obtención de clientes unificados

La petición **GET** a `/api/banco/clientes` invoca el método `listarTodosLosClientes()`, que consulta secuencialmente los tres nodos y concatena los resultados en un único arreglo JSON. La respuesta obtenida, con código **200 OK**, contiene los clientes de Cuenca (cédulas con prefijo 01), de Quito (prefijo 17) y de Guayaquil (prefijo 09) en una sola estructura, sin que el cliente HTTP tenga forma de distinguir, a partir de la respuesta, que los datos provienen de tres motores de base de datos físicamente independientes. Esto constituye la evidencia empírica de la transparencia de fragmentación: la tabla lógica `clientes` existe como concepto unificado solo en el momento en que el middleware realiza la unión de los fragmentos.

```
Dar formato al texto ☒
[
  {
    "cedula": "0101234567",
    "nombre": "Mario Vintimilla",
    "ciudad": "Cuenca"
  },
  {
    "cedula": "0107654321",
    "nombre": "Sofia Vasquez",
    "ciudad": "Cuenca"
  },
  {
    "cedula": "0103456789",
    "nombre": "Andres Cordero",
    "ciudad": "Cuenca"
  },
  {
    "cedula": "1701234567",
    "nombre": "Pedro Jimenez",
    "ciudad": "Quito"
  },
  {
    "cedula": "1709876543",
    "nombre": "Luisa Paredes",
    "ciudad": "Quito"
  },
  {
    "cedula": "1704567890",
    "nombre": "Carla Espinoza",
    "ciudad": "Quito"
  },
  {
    "cedula": "0900000001",
    "nombre": "Juan Pueblo",
    "ciudad": "Guayaquil"
  },
  {
    "cedula": "0900000002",
    "nombre": "Mafer Andrade",
    "ciudad": "Guayaquil"
  }
]
```

Figure 1: Respuesta de `/api/banco/clientes` con clientes de Cuenca, Quito y Guayaquil.

5.2 Prueba 2: Enrutamiento dinámico de saldos

Al invocar `GET /api/banco/saldo/{numero}` con distintos números de cuenta, el método `enrutar()` interpreta el prefijo y dirige la consulta exclusivamente al nodo correspondiente, sin tocar los otros dos motores. En la evidencia recolectada, la consulta sobre la cuenta 0901000002 (prefijo 09, Guayaquil) devuelve un JSON con saldo 1400.75 y oficina GUAYAQUIL en apenas 20 milisegundos, confirmando que la resolución del nodo destino es directa y no requiere una búsqueda previa en un directorio centralizado.

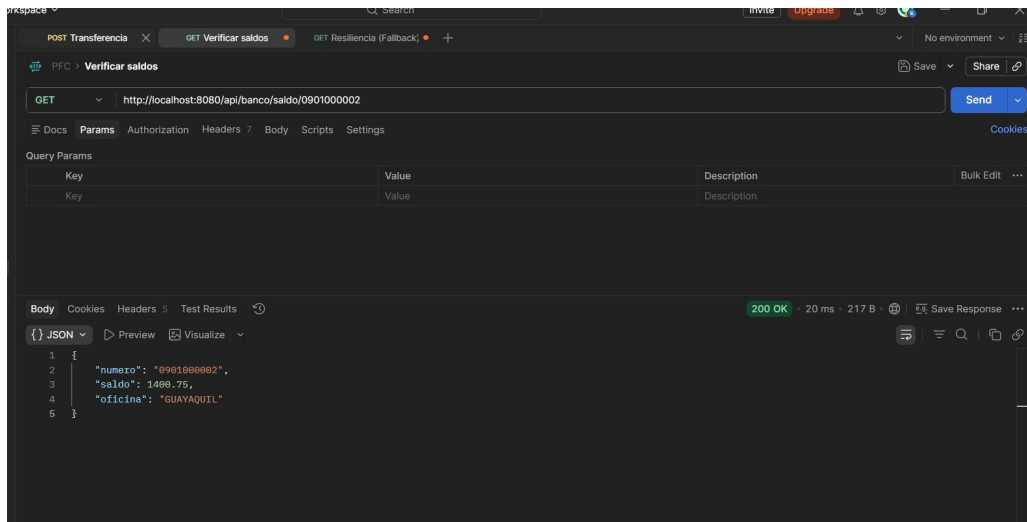


Figure 2: Consulta GET `/api/banco/saldo/0901000002` resuelta directamente contra el nodo Guayaquil.

5.3 Prueba 3: Transferencia inter-sede (Postman)

Esta prueba constituye la validación más exigente del prototipo, pues involucra dos nodos distintos dentro de una misma unidad transaccional. Se envió una petición POST a `/api/banco/transferencia` con el siguiente cuerpo JSON:

Cuerpo de la petición POST `/api/banco/transferencia`

```

{
  "origen": "0901000002",
  "destino": "1701000001",
  "monto": 100.00
}

```

El origen (0901000002) pertenece al nodo Guayaquil y el destino (1701000001) pertenece al nodo Quito, por lo que el método `realizarTransferencia()` resuelve dos `JdbcTemplate` distintos y determina que la operación es de tipo `INTER_SEDE`. La respuesta obtenida fue exitosa, con código 200 OK en 118 milisegundos:

Respuesta 200 OK

```

{
  "tipo": "INTER_SEDE (Distribuida)",
  "monto": 100.0,
  "mensaje": "Transferencia realizada con exito.",
  "origen": "0901000002",
  "destino": "1701000001"
}

```

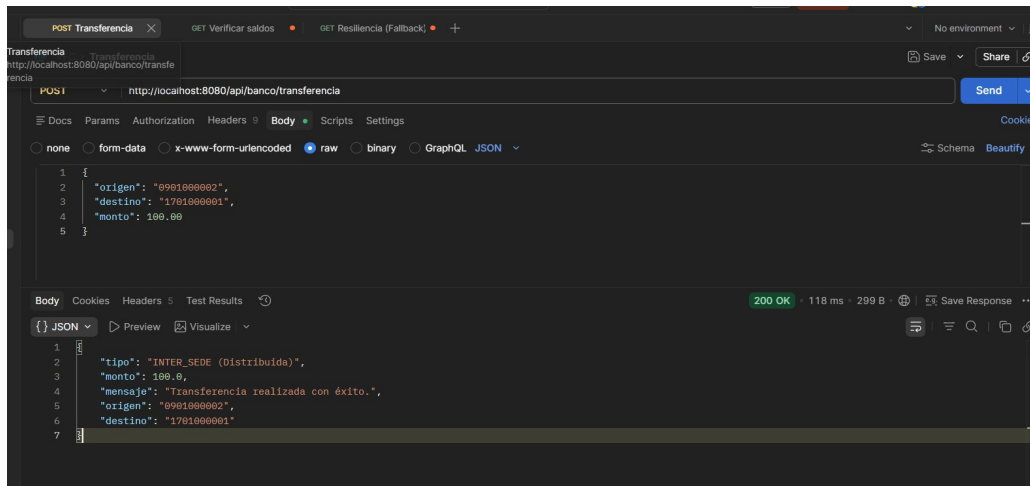


Figure 3: Transferencia inter-sede exitosa entre la cuenta Guayaquil y Quito.

La atomicidad de esta operación se garantiza mediante la anotación `@Transactional` de Spring: si el débito en el nodo de origen se ejecuta pero el crédito en el nodo destino lanzara una excepción (por ejemplo, porque la cuenta destino no existe), el framework revertiría el cambio ya aplicado dentro del contexto transaccional gestionado, evitando que el dinero desaparezca de la cuenta de origen sin acreditarse en ningún destino. Es importante precisar, sin embargo, una limitación técnica relevante para el análisis: dado que `dbOrigen` y `dbDestino` son `JdbcTemplate` construidos sobre `DataSource` físicamente distintos, la anotación `@Transactional` de Spring administra en rigor dos transacciones JDBC locales independientes y no una transacción distribuida verdadera coordinada por un gestor de transacciones globales (como XA/JTA). Esta distinción se retoma en la sección de trabajo futuro, donde se plantea el Commit en Dos Fases (2PC) como el mecanismo necesario para elevar esta garantía al nivel de producción.

5.4 Prueba 4: Tolerancia a fallos (resiliencia)

Para verificar el requisito de autonomía local, se detuvo deliberadamente el contenedor del nodo Quito mientras la aplicación Spring Boot permanecía en ejecución, mediante el comando:

```
docker stop banco-quito
```

Acto seguido se repitió la petición `GET /api/banco/clientes`. Gracias a los bloques `try-catch` independientes implementados en `listarTodosLosClientes()`, el sistema no colapsó con un error `500 Internal Server Error` como ocurriría con una implementación ingenua que propague la excepción de conexión rechazada. En su lugar, la aplicación respondió con código `200 OK` trayendo únicamente los clientes disponibles en los nodos vivos Cuenca y Guayaquil, mientras que en la consola del servidor se registró la advertencia interna **"Advertencia: Nodo QUITO fuera de línea."**, evidenciando de forma clara y auditable cuál componente experimentó la falla sin interrumpir el servicio para las sedes operativas. Al reiniciar el contenedor con `docker start banco-quito`, las peticiones subsiguientes volvieron a incluir a los tres nodos con normalidad, confirmando que la degradación fue temporal y selectiva.

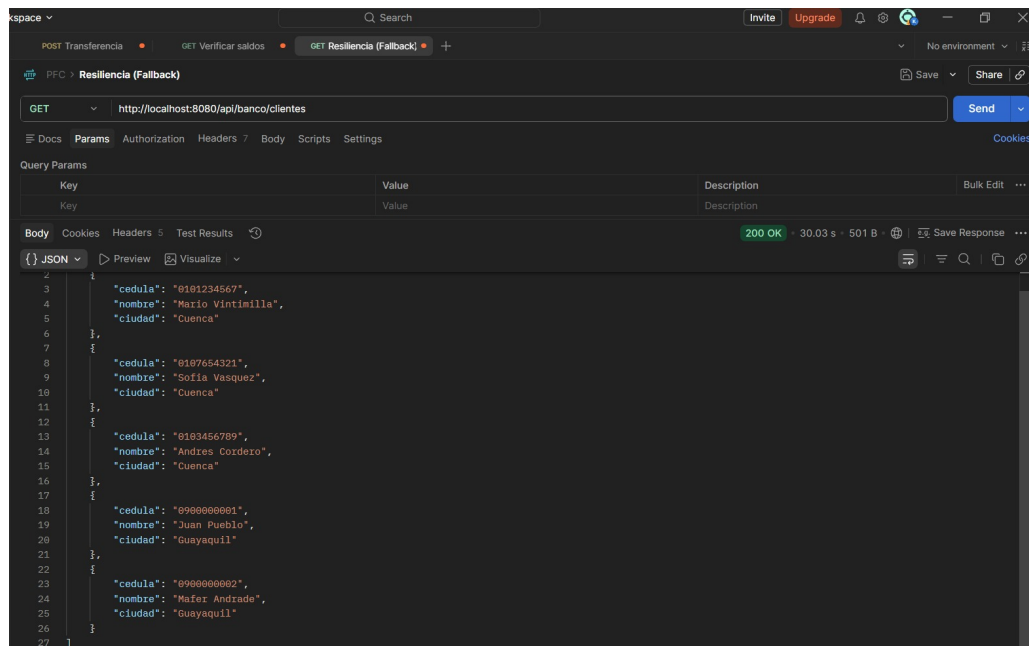


Figure 4: Fallback exitoso tras la caída simulada de un nodo

6. Análisis Teórico: Teorema de CAP y Modelo PACELC

6.1 Posicionamiento frente al teorema de CAP

El teorema de CAP, formulado por Eric Brewer, establece que ante una partición de red (P) es decir, la pérdida de comunicación entre nodos que en este prototipo se simuló deteniendo el contenedor de Quito un sistema distribuido debe elegir entre priorizar la Consistencia (C) o la Disponibilidad (A), sin poder garantizar ambas de forma absoluta durante el incidente. El prototipo del Banco del Austro se posiciona claramente como un sistema **CP** (Consistente y tolerante a Particiones): ante la caída del nodo Quito, la aplicación optó por no inventar ni aproximar datos de ese nodo, sino que simplemente los omitió de la respuesta, preservando la exactitud de la información entregada a costa de una disponibilidad parcial del catálogo completo de clientes. Un sistema AP habría intentado, en cambio, servir una copia potencialmente obsoleta de los datos de Quito desde alguna réplica de solo lectura, sacrificando consistencia por completitud de la respuesta.

6.2 El modelo PACELC: consistencia sobre disponibilidad

El modelo PACELC, propuesto por Daniel Abadi como una extensión del teorema de CAP, señala que incluso en ausencia de particiones de red (Else, E) un sistema debe decidir entre Latencia (L) y Consistencia (C). En el diseño del prototipo, esta decisión se resuelve consistentemente a favor de la consistencia: las transferencias inter-sede se ejecutan de forma síncrona dentro del contexto **@Transactional**, esperando la confirmación tanto del débito como del crédito antes de responder al cliente, en lugar de responder inmediatamente tras el débito y propagar el crédito de forma asíncrona o eventual. Esto añade latencia la operación de transferencia inter-sede tomó 118 ms frente a los 20 ms de una consulta de saldo local pero elimina la ventana de inconsistencia en la que el dinero estaría, desde la perspectiva de un observador externo, simultáneamente descontado y no acreditado.

6.3 Justificación del esquema de distribución por tabla

La decisión de replicar la tabla `clientes` y de fragmentar horizontalmente la tabla `cuentas` no es arbitraria, sino que responde directamente al patrón de acceso de cada entidad:

- **clientes (replicada):** cualquier oficina del banco debe poder identificar al titular de una cuenta por ejemplo, para validar su identidad en ventanilla sin importar en qué sede se originó ese cliente. Replicar esta tabla en los tres nodos evita una consulta remota costosa cada vez que se necesita el nombre de un titular, a cambio de un costo de escritura mayor cuando se registra un cliente nuevo, ya que en teoría debería propagarse a las tres sedes (limitación que se discute en la sección siguiente).
- **cuentas (fragmentada horizontalmente):** el saldo y los movimientos de una cuenta pertenecen, por naturaleza del negocio, a la oficina donde el cliente la abrió, y la inmensa mayoría de las consultas de saldo son locales a esa oficina. Fragmentar esta tabla evita duplicar el estado mutable más sensible del sistema el dinero en múltiples ubicaciones, lo que sería no solo costoso sino peligroso desde la perspectiva de integridad: mantener el saldo como una única fuente de verdad por cuenta, ubicada en un solo nodo, es lo que permite que las restricciones `CHECK` y las actualizaciones atómicas locales sean suficientes para garantizar la corrección del balance sin necesidad de un protocolo de consenso distribuido para cada operación.

7. Conclusiones

El prototipo demuestra, con evidencia empírica reproducible, que la transparencia de fragmentación y la transparencia de ubicación no son propiedades abstractas de la teoría de bases de datos distribuidas, sino comportamientos concretos que se materializan en el código de la capa middleware: el cliente HTTP consumió los mismos tres endpoints sin necesidad de conocer en ningún momento cuántos nodos PostgreSQL existían detrás, cuáles estaban activos, o en qué ciudad residía físicamente cada fragmento de datos. La estrategia de distribución adoptada replicación de la tabla `clientes` frente a fragmentación horizontal de la tabla `cuentas` confirma que el diseño de una base de datos distribuida no puede derivarse de una regla universal, sino que exige analizar, tabla por tabla, el patrón de acceso, la frecuencia de escritura y la sensibilidad del dato ante la inconsistencia, tal como se argumentó en el análisis teórico frente al modelo PACELC. La prueba de tolerancia a fallos evidenció que la resiliencia de un sistema distribuido no es una propiedad que emerja espontáneamente del uso de contenedores independientes, sino que debe programarse explícitamente: sin los bloques `try-catch` aislados por nodo implementados en `listarTodosLosClientes()`, la caída de un único contenedor habría degradado el servicio completo, contradiciendo el requisito de autonomía local que motivó la arquitectura distribuida desde su concepción. Si bien el prototipo alcanza satisfactoriamente sus objetivos didácticos incluyendo la extensión propuesta a un tercer nodo, el endpoint transaccional de transferencias y la prueba de resiliencia, el análisis honesto de sus limitaciones (transacciones JDBC locales en lugar de un 2PC verdadero, ausencia de Circuit Breaker, replicación manual de clientes) resulta tan valioso pedagógicamente como el propio funcionamiento exitoso del sistema, pues traza con precisión la brecha real que separa un prototipo de laboratorio de una base de datos distribuida de nivel productivo para el sector financiero.